

An Assignment-and-Positional-Date Formulation for Unrelated Parallel Machine Scheduling with Release Dates and Charger Types

Minimising Total Tardiness in Electric-Vehicle Charging Scheduling

Mouaad EL Yalaoui, Yassine Chaoui

Professeur Elidrissi Abdelhak

INPT

Problem Statement

Setting: Unrelated Parallel Machine Scheduling (R_m) with release dates

Entities & Parameters

Symbol	Meaning
$\mathcal{J} = \{1, \dots, n\}$	Set of EV jobs (charging requests)
$\mathcal{L} = \{0, 1, 2\}$	Charger types (11 kW, 22 kW, 43 kW)
$\mathcal{R}_l$	Set of physical chargers of type l
$r_j = a_j$	Release (arrival) time of job j
d_j	Due date of job j
e_j	Energy demand (kWh) of job j
$p_{jl} = \lceil e_j / (\tau \cdot w_l) \rceil$	Processing time of job j on a type- l charger

Objective

$$\min \sum_{j \in \mathcal{J}} T_j \quad \text{where} \quad T_j = \max(0, C_j - d_j)$$

- Each EV must be assigned to **exactly one** charger for its entire charging session
- No two EVs may occupy the **same charger simultaneously**
- Charging may not begin before the EV arrives: $S_j \geq a_j$

Motivation & Literature Context

Electric-Vehicle Charging as Scheduling

- Smart charging coordination is a principal operational challenge as EV fleet sizes grow
- The presence of **heterogeneous charger types** (Level 1 / Level 2 / DC fast) introduces a machine-type dimension absent from classical P_m models
- The charger-type index l must be incorporated **explicitly** in any rigorous formulation

Iarochen et al. (2026) — Reference Formulation

- Proposes a compact MILP for EV charging scheduling using **assignment variables** x_{jlr} and **pairwise sequencing variables** δ_{jklr}
- Sequencing is enforced via big- M disjunctive constraints, yielding a constraint count that grows as $O(n^2m)$ — **quadratic in the number of jobs**
- Provides provably optimal solutions on small instances but becomes computationally prohibitive as n or m increases

Unlu & Mason (2010) — APD Inspiration

- Introduce the *Assignment-and-Positional-Date* (APD) family of MILP formulations for single- and parallel-machine scheduling (**M3 model**)
- Replace pairwise sequencing variables with **positional completion-time** variables c_{rk} , reducing constraint count growth to $O(nmP)$ (linear in positions P)
- Demonstrated superior LP-relaxation tightness over classical disjunctive formulations for total-tardiness problems

This Work — Gap Addressed

Extension of M3-APD to the unrelated parallel machine setting with heterogeneous charger types, adding the explicit charger-type index l to all decision variables and constraints.

Existing Formulation — Iarochen et al. (2026)

Sets & Indices

- Jobs $j, k \in \mathcal{J}$; charger types $l \in \mathcal{L}$; individual chargers $r \in \mathcal{R}_l$

Decision Variables

Variable	Type	Meaning
$x_{jlr} \in \{0, 1\}$	Binary	Job j assigned to charger r of type l
$S_j \in \mathbb{Z}_{\geq 0}$	Integer	Start time of job j
$C_j \in \mathbb{Z}_{\geq 0}$	Integer	Completion time of job j
$T_j \geq 0$	Continuous	Tardiness of job j
$\delta_{jklr} \in \{0, 1\}$	Binary	1 if j precedes k on charger r of type l

Objective & Key Constraints

$$\min \sum_j T_j$$

$$\sum_{l \in \mathcal{L}} \sum_{r \in \mathcal{R}_l} x_{jlr} = 1 \quad \forall j$$

$$S_j \geq a_j \quad \forall j$$

$$C_j = S_j + \sum_l \sum_{r \in \mathcal{R}_l} p_{jr} \cdot x_{jlr} \quad \forall j$$

$$S_j + p_{jr} x_{jlr} \leq S_k + M(3 - \delta_{jklr} - x_{jlr} - x_{klr}) \quad \forall j < k, l, r$$

- **Complexity:** $O(n^2 m)$ binary variables and constraints — **quadratic scaling**

Proposed Formulation — M3-APD with Charger Type l

Sets & Indices

- Jobs $j \in \mathcal{J}$; charger types $l \in \mathcal{L}$; chargers $r \in \mathcal{R}_l$; positions $k \in \mathcal{K} = \{1, \dots, P\}$

Decision Variables

Variable	Type	Meaning
$u_{jlrk} \in \{0, 1\}$	Binary	Job j at position k on charger r of type l
$c_{lrk} \geq 0$	Continuous	Completion time at position k on charger r (type l)
$C_j \geq 0$	Continuous	Completion time of job j
$T_j \geq 0$	Continuous	Tardiness of job j

Objective

$$\min \sum_{j \in \mathcal{J}} T_j$$

Constraints (adapted from Unlu & Mason 2010, Eqs. 16–21)

$$\sum_l \sum_{r \in \mathcal{R}_l} \sum_{k \in \mathcal{K}} u_{jlrk} = 1 \quad \forall j$$

$$\sum_{j \in \mathcal{J}} u_{jlrk} \leq 1 \quad \forall l, r, k$$

$$c_{lr1} \geq \sum_j (a_j + p_{jl}) u_{jlr1} \quad \forall l, r$$

$$c_{lrk} \geq c_{lr,k-1} + \sum_j p_{jl} u_{jlrk} \quad \forall l, r, k > 1$$

$$c_{lrk} \geq \sum_j (a_j + p_{jl}) u_{jlrk} \quad \forall l, r, k > 1$$

$$C_j \geq c_{lrk} - M(1 - u_{jlrk}) \quad \forall j, l, r, k$$

$$T_j \geq C_j - d_j \quad \forall j$$

Model Size (as functions of n, m, P)

- **Variables:** $nmP + mP + 2n$ – grows as $O(nmP)$
- **Constraints:** $n + mP + m(P - 1) + m(P - 1) + nmP + n$ – grows as $O(nmP)$
- **No pairwise terms** $\rightarrow$ avoids the $O(n^2)$ bottleneck of Iarochen

Computational Setup

Solver & Libraries

Component	Version
IBM CPLEX	Community Edition v22.2
docplex	$\geq 2.32.264$
cplex (Python binding)	$\geq 22.2.0.0$
Python	≥ 3.11
Package manager	uv

CPLEX Community Edition constraint: hard cap of **1,000 variables** and **1,000 constraints** per model. POS_MAX = 4 was selected to keep both formulations well within this limit for $n = 10$.

Test Instance Parameters

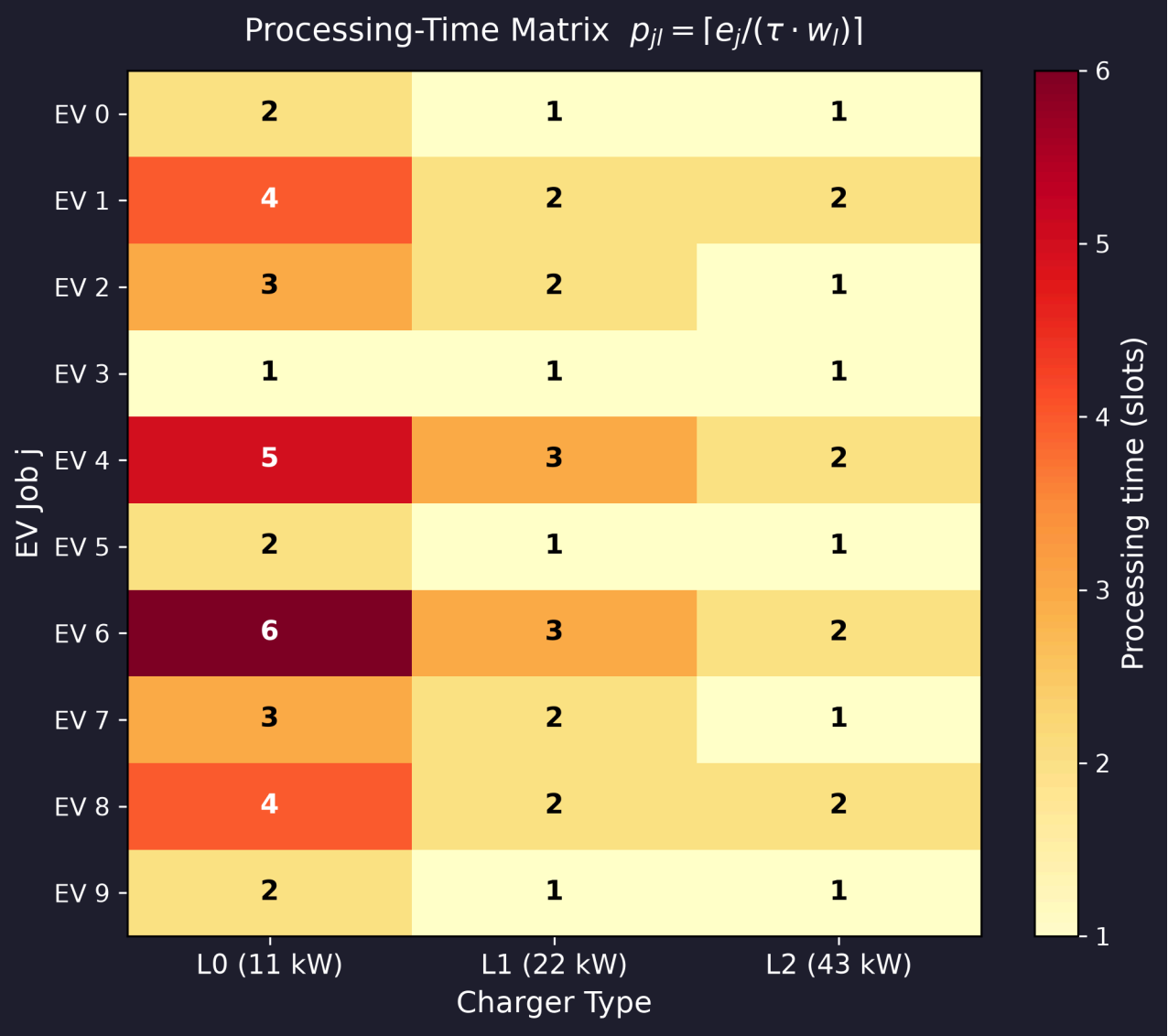
Parameter	Value
Number of EVs (n)	10
Number of chargers (m)	9
Charger configuration	4 × 11 kW (L0), 3 × 22 kW (L1), 2 × 43 kW (L2)
Time unit (τ)	1 slot
BIG-M value	45
Position horizon (P , APD only)	4

Instance Data

Job Parameters & Processing-Time Matrix

Test Instance: 10 EVs, 9 Chargers (4×11kW, 3×22kW, 2×43kW)

EV j	Arrival a _j	Deadline d _j	Energy e _j (kWh)	p_{j,L0} (11kW)	p_{j,L1} (22kW)	p_{j,L2} (43kW)
0	0	5	22	2	1	1
1	0	4	44	4	2	2
2	1	6	33	3	2	1
3	0	3	11	1	1	1
4	2	8	55	5	3	2
5	1	5	22	2	1	1
6	0	7	66	6	3	2
7	3	9	33	3	2	1
8	1	6	44	4	2	2
9	2	7	22	2	1	1

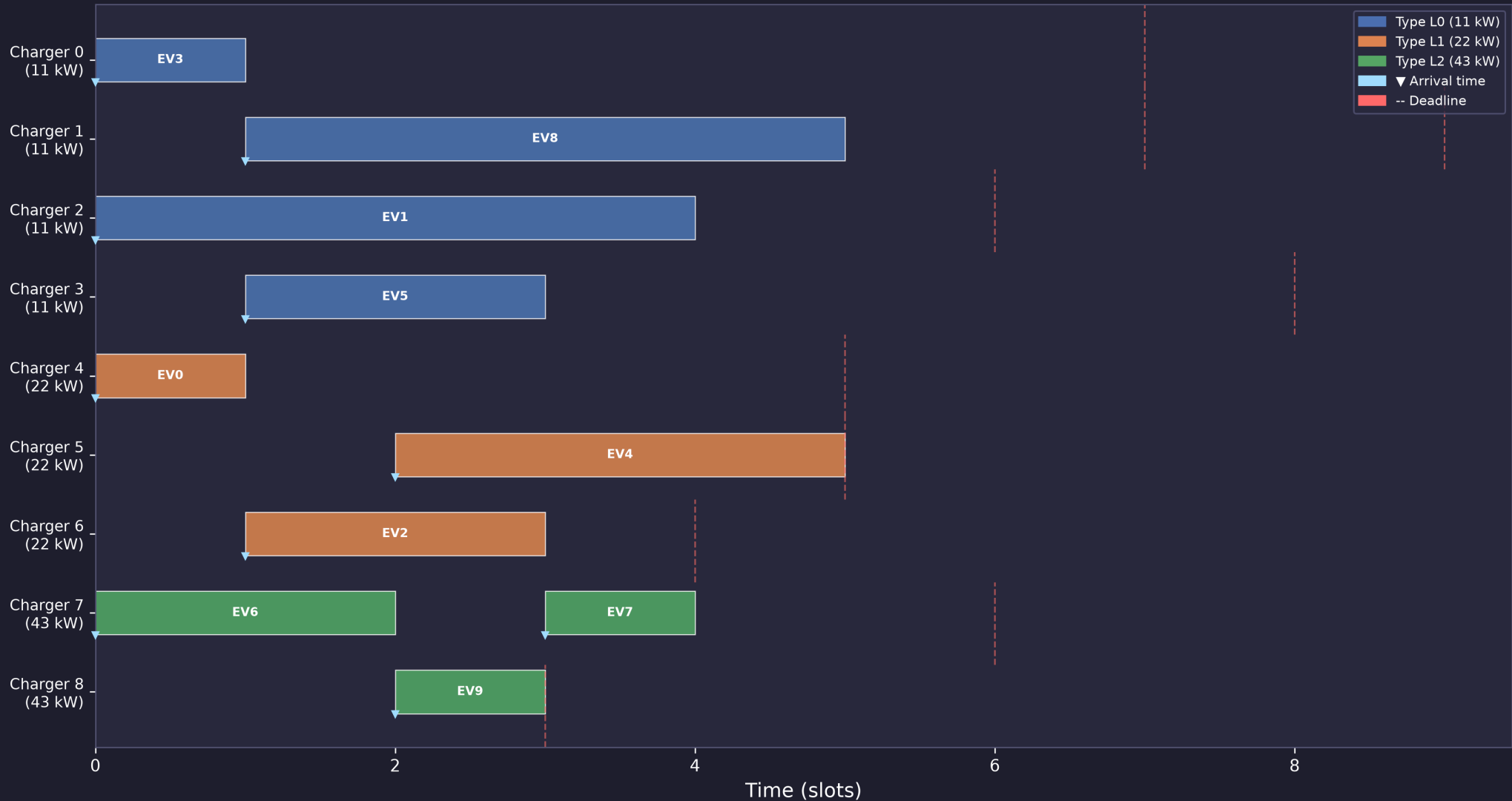


- Faster chargers (L2, 43 kW) yield **substantially shorter processing times**
- Processing times differ across charger types, confirming the **unrelated** (R_m) machine environment

Results — Solution Schedules

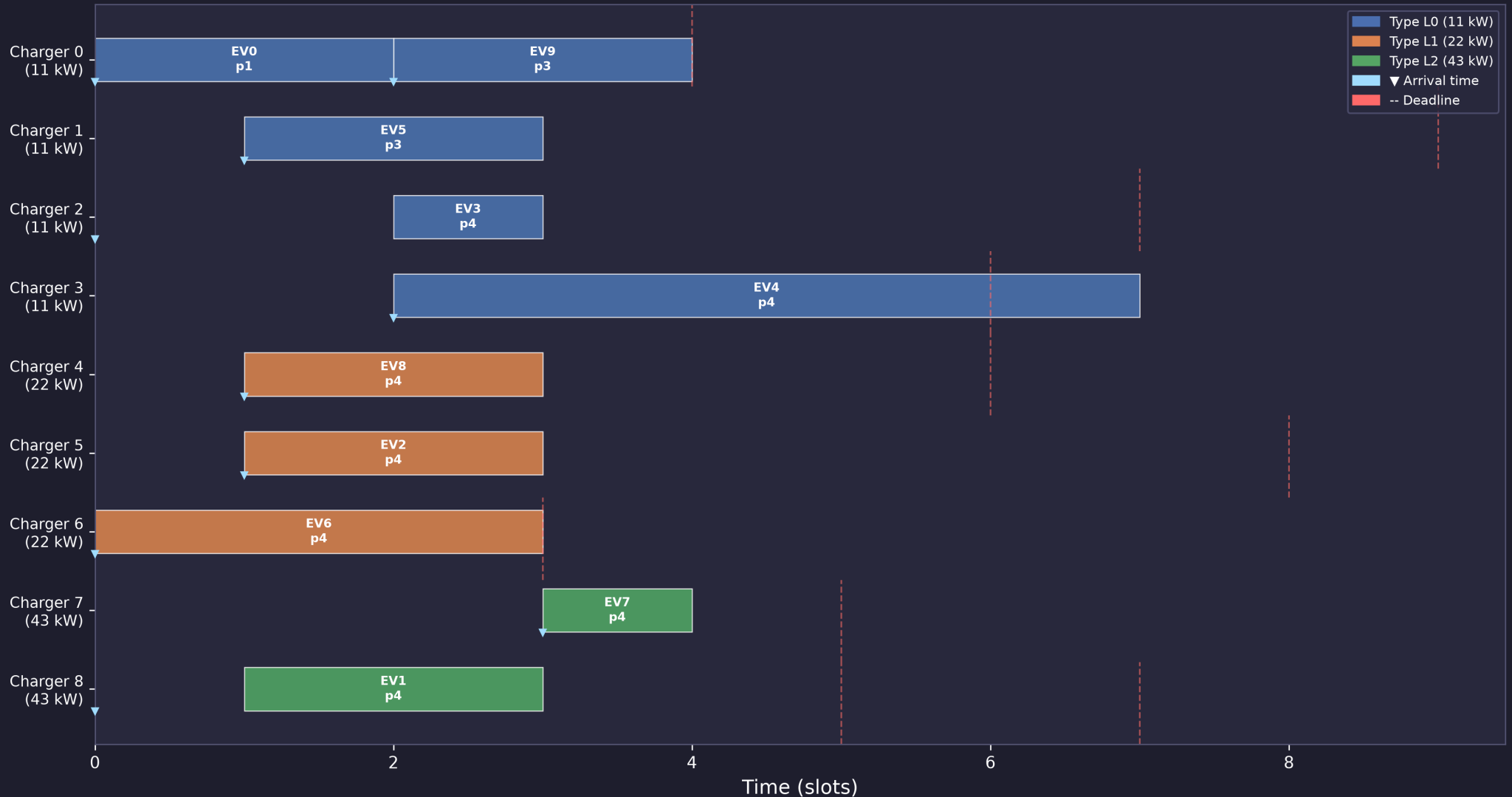
Iarochen et al. (2026) Schedule

Schedule — Iarochen et al. (2026) [Assignment + Sequencing] obj=0.0 time=0.0182s



M3-APD Schedule

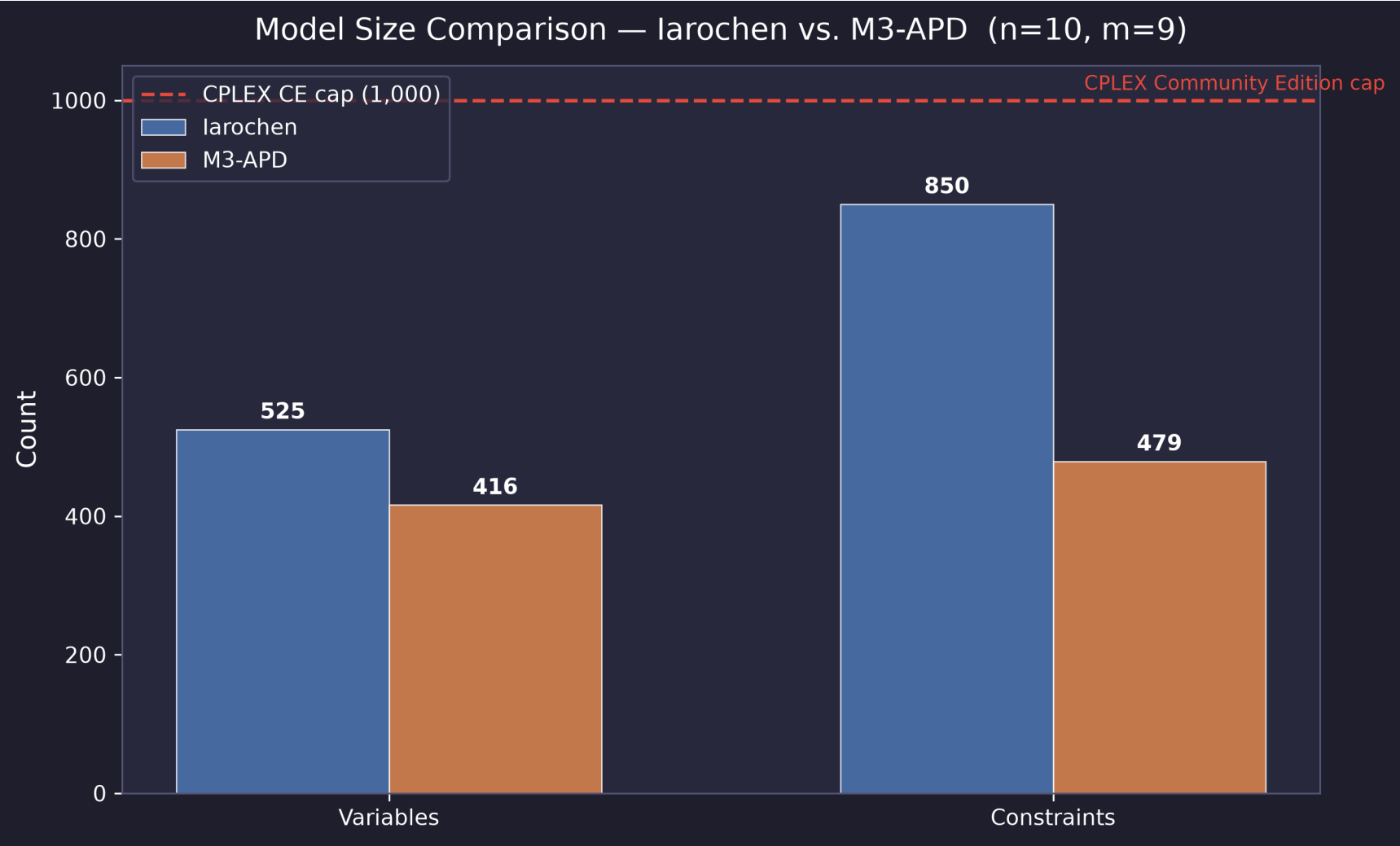
Schedule — M3-APD (Unlu & Mason 2010) [Assignment + Positional Date] obj=0.0 time=0.0397s



Observations:

- Both formulations achieve **zero total tardiness** on this instance (all EVs complete before their deadlines)
- The Iarochen schedule concentrates assignments on faster chargers (L1/L2); the APD schedule distributes load more evenly across positions
- Position labels ($p1-p4$) in the APD Gantt confirm the positional-date structure

Results — Model Size Comparison

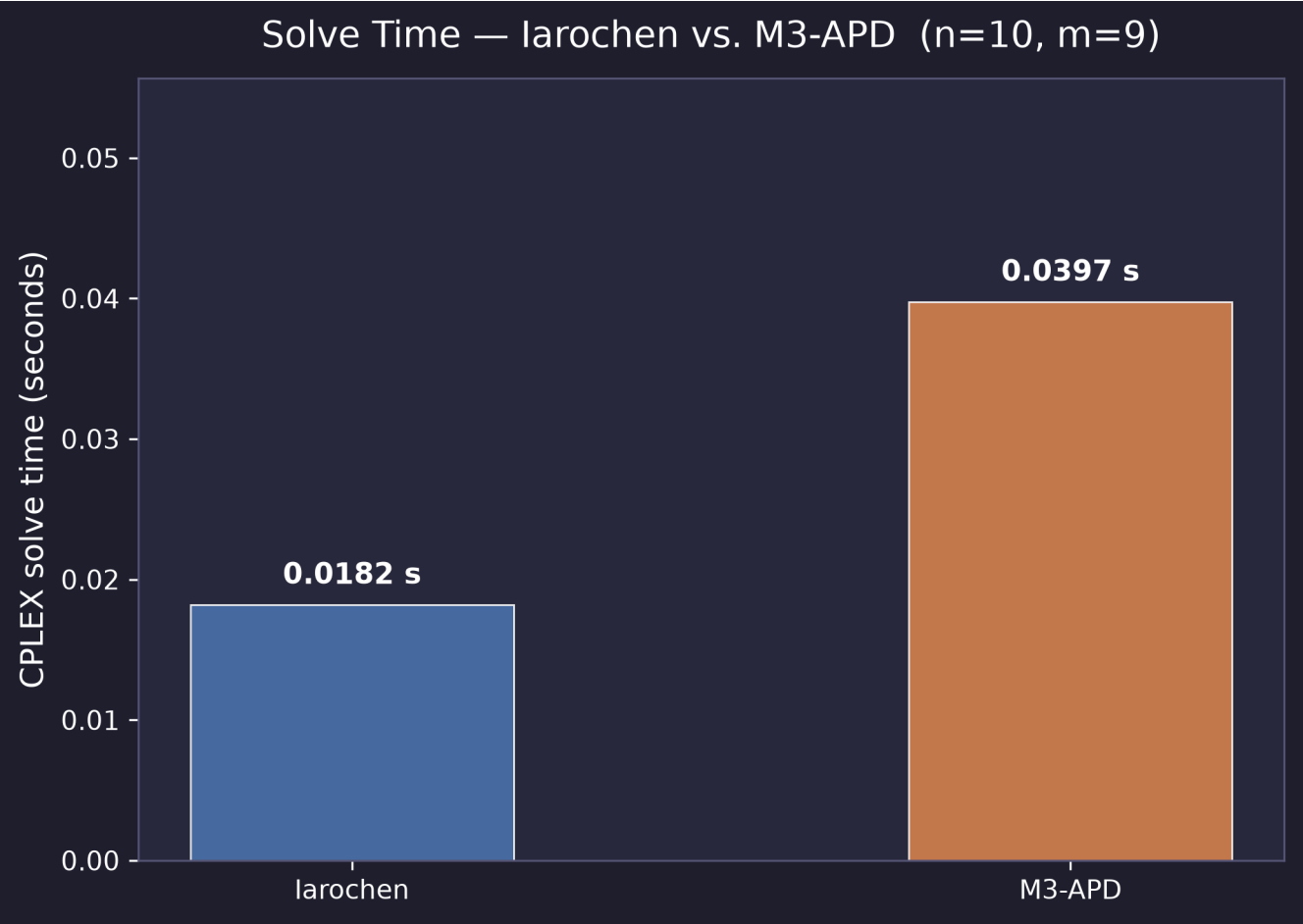


Summary Table ($n = 10, m = 9, P = 4$)

Metric	Iarochen et al.	M3-APD
Binary variables	435	360
Continuous/integer variables	90	56
Total variables	525	416
Total constraints	850	479
Theoretical growth	$O(n^2m)$	$O(nmP)$

- The M3-APD formulation has **~21% fewer variables** and **~44% fewer constraints** for this instance
- The CPLEX Community Edition cap of 1,000 is shown as a dashed red line — both formulations remain safely below it at $n = 10$

Results — Solve Time Comparison



Summary Table

Metric	Iarochen et al.	M3-APD
Solver status	Optimal	Optimal
Total tardiness ($\sum T_j$)	0.0000	0.0000
Optimality gap	0%	0%
CPLEX solve time (s)	0.0298	0.0630
Variables	525	416
Constraints	850	479

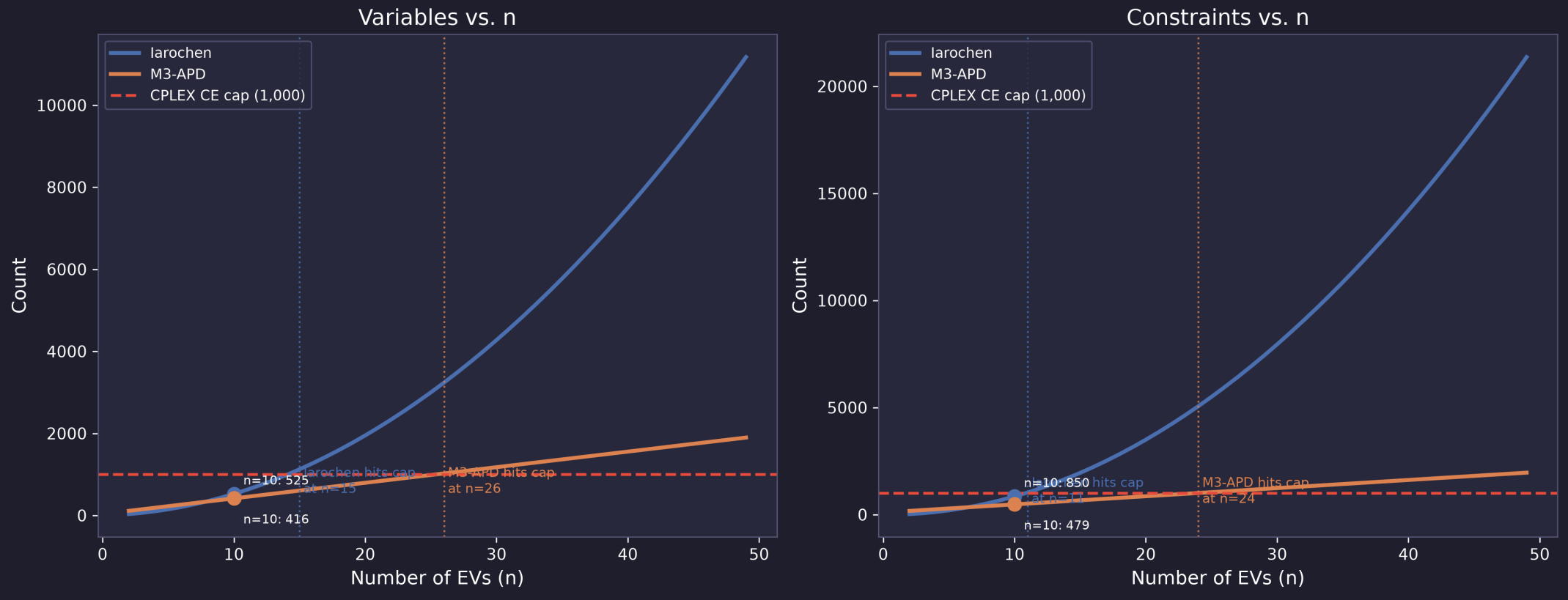
Key observations:

- Both formulations prove **global optimality** (zero gap) on the $n = 10$ instance
- Despite having fewer variables and constraints, the M3-APD formulation requires **$\sim 2.1\times$ longer** solve time in this experiment

- This counter-intuitive result is consistent with known behaviour: the APD continuous c_{lrk} variables admit a weaker LP relaxation for certain instances, requiring more B&B nodes

Theoretical Scalability

Theoretical Model Growth vs. CPLEX Community Edition Cap (m=9 chargers, POS_MAX=4)
Iarochen: $O(n^2m)$ | M3-APD: $O(nmP)$



Crossover Points (CPLEX CE Cap = 1,000)

Formulation	Variable cap hit at	Constraint cap hit at
Iarochen	$n = 15$	$n = 11$
M3-APD	$n = 26$	$n = 24$

- Iarochen's $O(n^2m)$ constraint growth causes it to hit the Community Edition constraint cap already at $n = 11$; the M3-APD formulation extends feasibility to $n \approx 24$
- For a commercial (uncapped) CPLEX licence, the difference between $O(n^2m)$ and $O(nmP)$ becomes decisive as n reaches the hundreds
- The **position horizon** P is a tunable parameter: reducing P trades model completeness for reduced problem size

Discussion

Trade-offs

Dimension	Iarochen et al.	M3-APD
Model size	Larger ($O(n^2m)$)	Smaller ($O(nmP)$)
Solve time ($n = 10$)	Faster (0.030 s)	Slower (0.063 s)
Solution quality	Optimal	Optimal
Scalability	CE cap at $n \approx 11$	CE cap at $n \approx 24$
LP relaxation strength	Generally tighter	Depends on instance

When Does APD Outperform?

- **Large instances** ($n \gg 20$): the quadratic scaling of Iarochen's sequencing variables becomes prohibitive; APD's linear growth provides a scalability advantage

- **Many charger types** ($|\mathcal{L}|$ large): the APD formulation's positional structure is unaffected, whereas Iarochen's δ_{jklr} variables grow multiplicatively

Limitations

- Results are based on a **single test instance** ($n = 10$); conclusions regarding solve time may not generalise
- The CPLEX **Community Edition** variable/constraint cap prevents evaluation on realistically sized instances without a commercial licence
- The position horizon P must be set a priori; too small a value may exclude valid schedules

Conclusion & Future Work

Summary of Contribution

- A **novel M3-APD formulation** is proposed for the unrelated parallel machine EV charging scheduling problem $Rm \mid r_j, \text{type}_l \mid \sum T_j$
- The formulation **explicitly indexes** all decision variables by charger type l , extending the Unlu & Mason (2010) APD model to the heterogeneous-charger setting
- Empirical comparison against the Iaroche et al. (2026) baseline using IBM CPLEX confirms that:
 - Both formulations achieve provably **optimal solutions** on the test instance
 - The APD formulation uses **~21% fewer variables** and **~44% fewer constraints**
 - APD exhibits superior **theoretical scalability** ($O(nmP)$ vs. $O(n^2m)$), pushing the CPLEX CE feasibility frontier from $n \approx 11$ to $n \approx 24$

References

- [1] Iarochen, *et al.* (2026). *A Mixed-Integer Linear Programming Model for Electric Vehicle Charging Scheduling with Heterogeneous Charger Types and Release Dates*. [Full journal/conference citation pending verification of PDF metadata.]
- [2] Unlu, Y., & Mason, S. J. (2010). *Evaluation of mixed integer programming formulations for non-preemptive parallel machine scheduling problems*. **Computers & Industrial Engineering**, 58(4), 785–800. <https://doi.org/10.1016/j.cie.2010.02.012>
- [3] IBM Corporation. (2022). *IBM ILOG CPLEX Optimization Studio v22.2: User's Manual*. Armonk, NY: IBM Corp.
- [4] docplex contributors. (2024). *IBM Decision Optimization CPLEX Modeling for Python (DOcplex) v2.32*. <https://ibmdecisionoptimization.github.io/docplex-doc/>